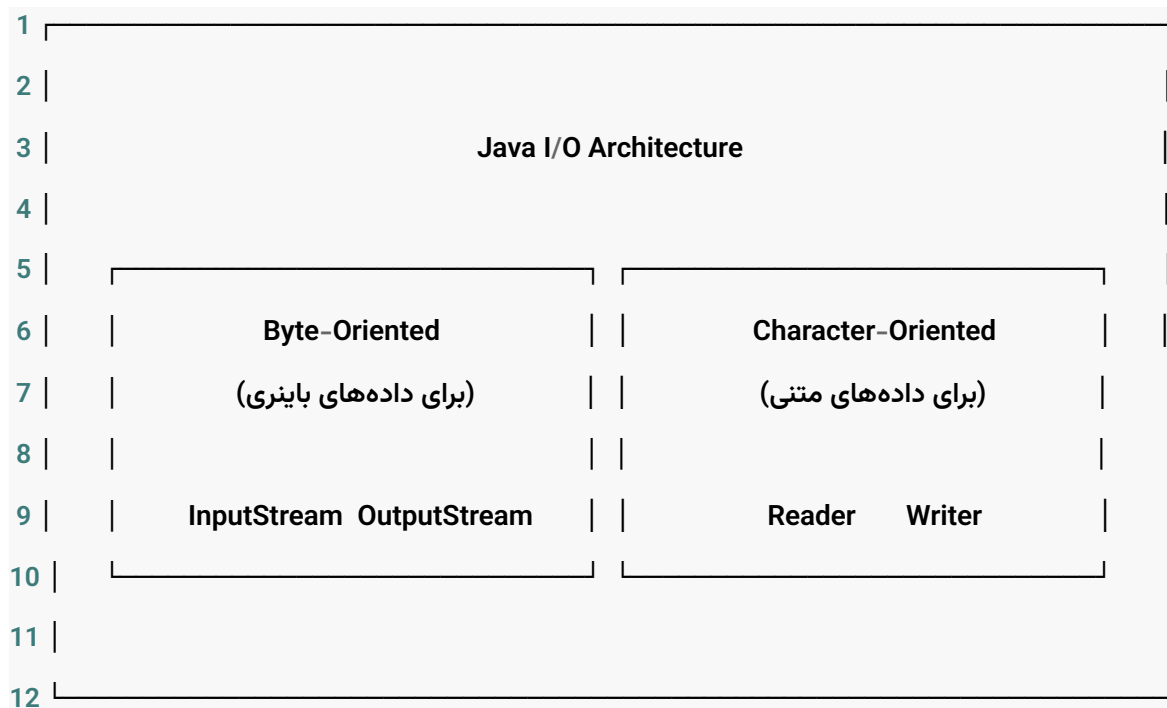


فاز ۱.۵: ورودی/خروجی (I/O) در جاوا – کامل و عمیق

۱.۵.۱ مقدمه: I/O در جاوا چیست؟

جاوا برای کار با ورودی/خروجی (Input/Output) دو دسته کلی دارد:



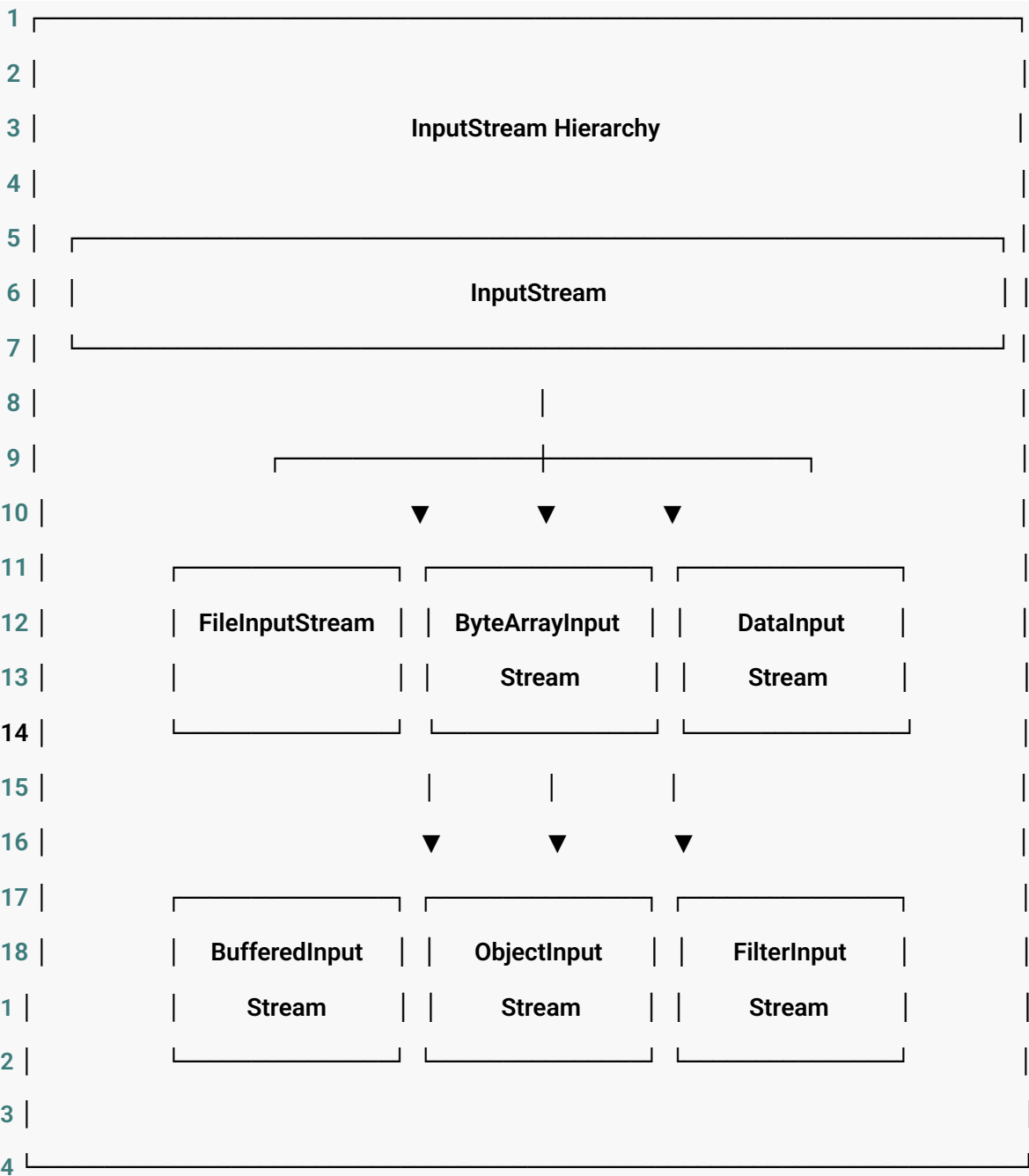
فرق اصلی:

• Stream (بایت): برای فایل‌های باینری (تصویر، ویدیو، صدا، PDF)

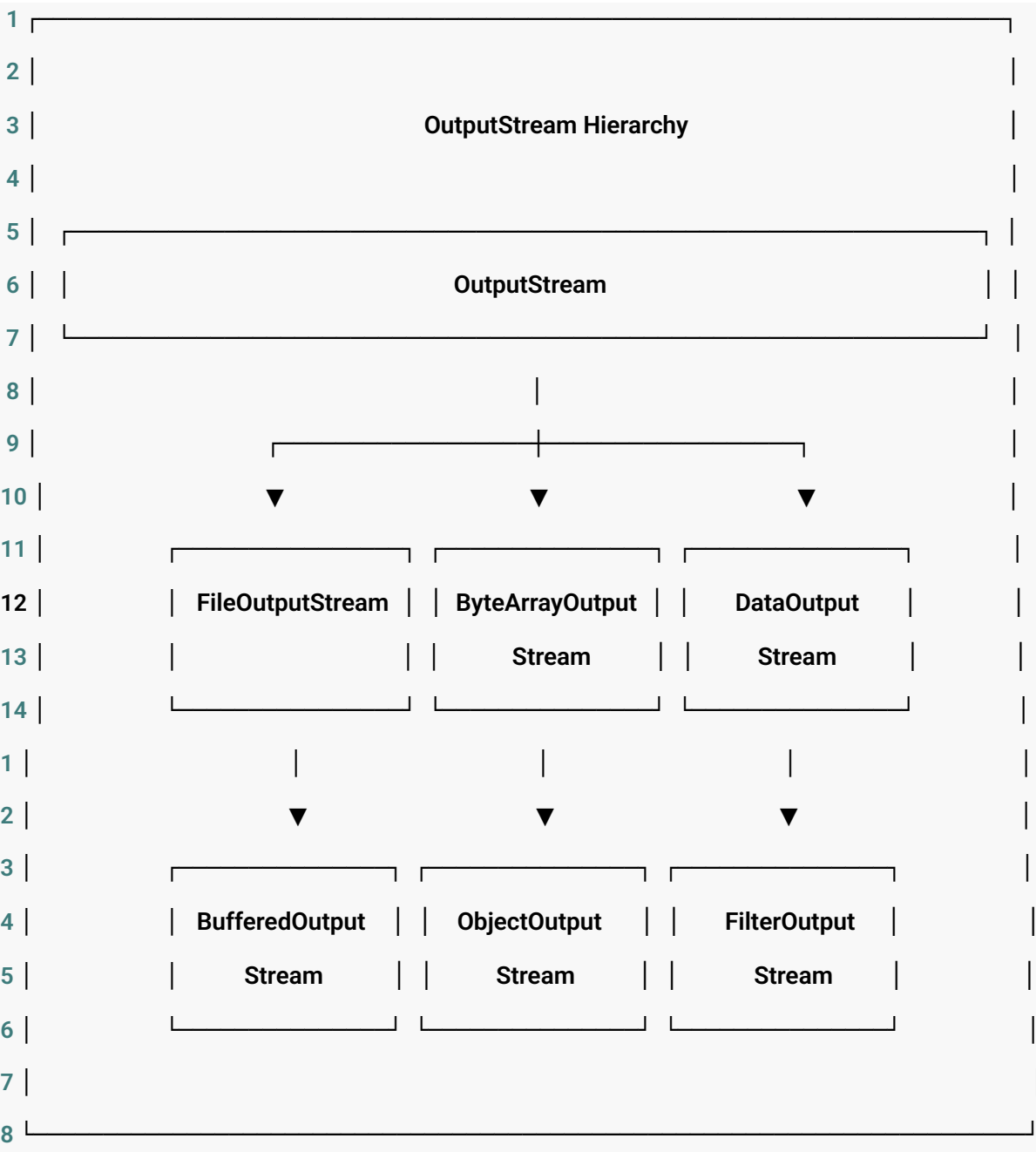
• Reader/Writer (کاراکتر): برای فایل‌های متنی (TXT, CSV, JSON, XML)

1.5.2 Stream های بایت (Byte Streams)

InputStream (خواندن داده های باینری)



OutputStream (نوشتن داده‌های باینری)



مثال‌های عملی با Byte Streams

۱. FileInputStream (خواندن فایل)

```

1 // a
2 // روش ۱: خواندن بایت به بایت
3 try (FileInputStream fis = new FileInputStream("image.jpg")) {
4     int byteData;
5     while ((byteData = fis.read()) != -1) {
6         System.out.print(byteData); // مقادیر 0-255
7     }
8 } catch (IOException e) {
9     e.printStackTrace();
10 }
11
12 // روش ۲: خواندن تکه تکه (بهتر)
13 try (FileInputStream fis = new FileInputStream("image.jpg")) {
14     byte[] buffer = new byte[1024]; // KB بافر ۱
15     int bytesRead;
16     while ((bytesRead = fis.read(buffer)) != -1) {
17         System.out.println("بایت " + bytesRead + ": خوانده شد");
18         // buffer[0] تا buffer[bytesRead-1] پردازش
19     }
20 } catch (IOException e) {
21     e.printStackTrace();
22 }

```

```

1 // b
2 // نوشتن بایت‌ها در فایل
3 try (FileOutputStream fos = new FileOutputStream("output.jpg")) {
4     String text = "Hello World!";
5     byte[] data = text.getBytes();
6     fos.write(data);
7     System.out.println("فایل ذخیره شد");
8 } catch (IOException e) {
9     e.printStackTrace();
10 }
11
12 // کپی کردن فایل
13 try (FileInputStream fis = new FileInputStream("source.jpg");
14     FileOutputStream fos = new FileOutputStream("dest.jpg")) {
15     byte[] buffer = new byte[4096];
16     int bytesRead;
17     while ((bytesRead = fis.read(buffer)) != -1) {
18         fos.write(buffer, 0, bytesRead);
19     }
20     System.out.println("فایل کپی شد");
21 } catch (IOException e) {
22     e.printStackTrace();
23 }

```

۳. BufferedInputStream/OutputStream (بهبود سرعت)

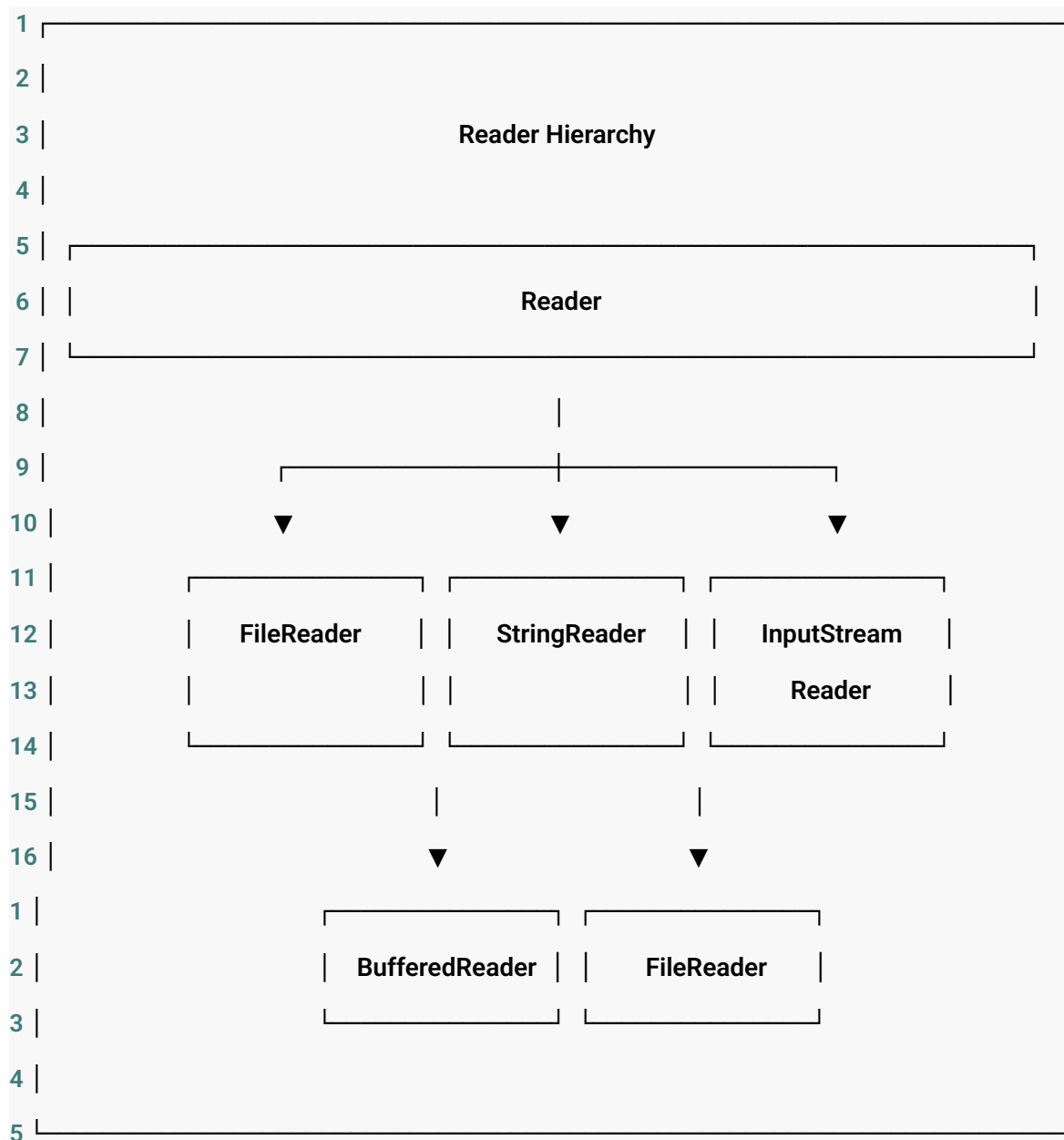
```

1 // c
2 با بافر - خیلی سریع‌تر //
3 try (BufferedInputStream bis = new BufferedInputStream(
4     new FileInputStream("large_file.zip"));
5     BufferedOutputStream bos = new BufferedOutputStream(
6         new FileOutputStream("copy.zip"))) {
7
8     byte[] buffer = new byte[8192];
9     int bytesRead;
10    while ((bytesRead = bis.read(buffer)) != -1) {
11        bos.write(buffer, 0, bytesRead);
12    }
13    System.out.println("فایل با بافر کپی شد");
14 } catch (IOException e) {
15     e.printStackTrace();
16 }

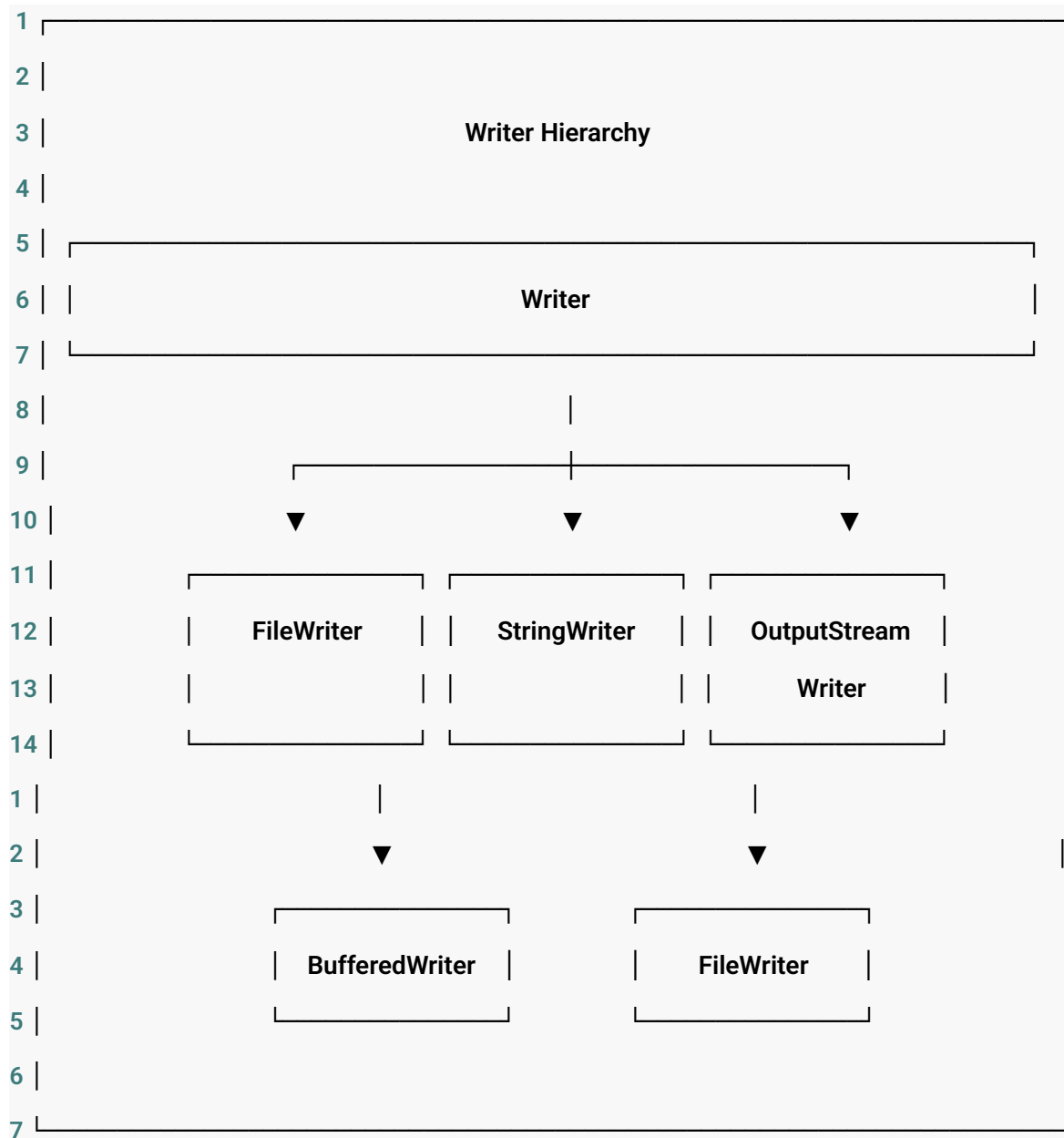
```

۳.۱.۵ Reader و Writer (داده‌های متنی)

Reader (خواندن متن)



Writer (نوشتن متن)



مثال‌های عملی با Reader/Writer

۱. FileReader (خواندن فایل متنی)

```

1 // a
2 // روش ۱: کاراکتر به کاراکتر (کند)
3 try (FileReader fr = new FileReader("file.txt")) {
4     int charData;
5     while ((charData = fr.read()) != -1) {
6         System.out.print((char) charData);
7     }
8 } catch (IOException e) {
9     e.printStackTrace();
10 }

```

۲. BufferedReader (توصیه شده)

```

1 // b
2 // روش ۲: BufferedReader - خط به خط (سریع‌تر)
3 try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
4     String line;
5     while ((line = br.readLine()) != null) {
6         System.out.println(line);
7     }
8 } catch (IOException e) {
9     e.printStackTrace();
10 }

```

```

9
10 // روش ۳: با مشخص کردن کدگذاری
11 try (BufferedReader br = new BufferedReader(
12     new InputStreamReader(new FileInputStream("file.txt"), "UTF-8"))) {
13     String line;
14     while ((line = br.readLine()) != null) {
15         System.out.println(line);
16     }
17 } catch (IOException e) {
18     e.printStackTrace();
19 }

```

۳. FileWriter (نوشتن فایل متنی)

```

1 // c
2 // روش ۱: ساده FileWriter
3 try (FileWriter fw = new FileWriter("output.txt")) {
4     fw.write("Hello World!\n");
5     fw.write("خط دوم");
6 } catch (IOException e) {
7     e.printStackTrace();
8 }
9
10 // روش ۲: توصیه شده BufferedWriter
11 try (BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"))) {
12     bw.write("Hello World!");
13     bw.newLine();
14 }

```

```

9  bw.write("خط دوم");
10 bw.write("و خط سوم");
11 } catch (IOException e) {
12     e.printStackTrace();
13 }
14
15 // اضافه کردن به انتهای فایل (Append روش ۳: با)
16 try (BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt", true))) {
17     bw.write("این به انتها اضافه میشه");
18     bw.newLine();
19 } catch (IOException e) {
20     e.printStackTrace();
21 }

```

۴. PrintWriter (نوشتن با فرمت)

```

1 // d
2 // PrintWriter - شبیه System.out
3 try (PrintWriter pw = new PrintWriter(new FileWriter("output.txt"))) {
4     pw.println("Hello World!");
5     pw.printf("نام: %s, سن: %d%n", "علی", 25);
6     pw.print("newline خط بدون");
7     pw.flush(); // فرستادن به فایل
8 } catch (IOException e) {
9     e.printStackTrace();
10 }

```

۱.۵.۴ Java NIO (New I/O) - مدرن تر

(*) java.nio از Java 1.4 اضافه شد و راه‌های مدرن‌تری برای I/O ارائه می‌دهد.

مقایسه IO و NIO

مقایسه IO و NIO در جاوا

ویژگی	IO (java.io)	NIO (java.nio)
جهت‌گیری	Stream-oriented (یک‌طرفه)	Buffer-oriented (دوطرفه)
بلوک‌شدن	Blocking (مسدود)	Non-blocking (غیرمسدود)
انتخاب‌گر	ندارد	Selector (چند کانال)
مناسب برای	فایل‌های معمولی	سرورهای با ترافیک بالا
API	ساده‌تر	پیچیده‌تر ولی قدرتمندتر

مثال‌های NIO

```

1 import java.nio.file.*;
2 import java.nio.*;
3
4 // ۱. خواندن همه خطوط (راحت‌ترین!)
5 List<String> lines = Files.readAllLines(Paths.get("file.txt"));
6 lines.forEach(System.out::println);
7
8 // ۲. نوشتن (راحت‌ترین!)
9 List<String> content = Arrays.asList("Hello", "World", "Line 3");
10 Files.write(Paths.get("output.txt"), content, StandardCharsets.UTF_8);

```

```
11
12 // ۳. کپی کردن فایل
13 Files.copy(Paths.get("source.txt"), Paths.get("dest.txt"),
14     StandardCopyOption.REPLACE_EXISTING);
15
16 // ۴. حذف فایل
17 Files.deleteIfExists(Paths.get("temp.txt"));
18
19 // ۵. چک کردن وجود فایل
20 if (Files.exists(Paths.get("file.txt"))) {
21     System.out.println("فایل وجود دارد");
22 }
23
24 // ۶. خواندن کل محتوا به صورت String (Java 11+)
25 String content = Files.readString(Paths.get("file.txt"));
26 System.out.println(content);
27
28 // ۷. نوشتن String به فایل (Java 11+)
29 Files.writeString(Paths.get("output.txt"), "Hello World!",
30     StandardCharsets.UTF_8);
31
32 // ۸. لیست کردن فایل‌های یک پوشه
33 try (Stream<Path> paths = Files.list(Paths.get("."))) {
34     paths.forEach(path -> System.out.println(path.getFileName()));
35 }
36
```

```

12 // ۹. ایجاد پوشه
13 Files.createDirectories(Paths.get("myapp/temp/logs"));
14
15 // ۱۰. دریافت اطلاعات فایل
16 Path path = Paths.get("file.txt");
17 System.out.println("نام: " + path.getFileName());
18 System.out.println("سایز: " + Files.size(path) + " بایت");
19 System.out.println("آخرین تغییر: " + Files.getLastModifiedTime(path));
20 System.out.println("نوع: " + Files.probeContentType(path));

```

۱.۵.۵ Object Serialization (ذخیره سازی اشیاء)

Serialization یعنی تبدیل یک شیء به جریان بایت برای ذخیره یا انتقال.

قوانین Serialization

- کلاس باید Serializable پیاده سازی کنه
- فیلدهای transient ذخیره نمیشن
- فیلدهای static ذخیره نمیشن
- کلاس باید serialVersionUID داشته باشه

مثال کامل

```

1 import java.io.*;
2
3 // ۱. کلاس Serializable

```

```

4 class Person implements Serializable {
5     private static final long serialVersionUID = 1L;
6
7     private String name;
8     private int age;
9     private transient String password; // ذخیره نمیشه
10    private static String company = "ACME"; // ذخیره نمیشه
11
12    public Person(String name, int age, String password) {
13        this.name = name;
14        this.age = age;
15        this.password = password;
16    }
17
18    // getters and setters
19
20
21    // ۲. Serialize (ذخیره)
22
23    public class SerializeExample {
24        public static void main(String[] args) {
25            Person person = new Person("Ali", 25, "secret123");
26
27            try (ObjectOutputStream oos = new ObjectOutputStream(
28                new FileOutputStream("person.ser"))) {
29                oos.writeObject(person);
30                System.out.println("شئ ذخیره شد");
31            }
32        }
33    }

```

```

17     } catch (IOException e) {
18         e.printStackTrace();
19     }
20 }
21 }
22
23 // ۳. Deserialize (بازیابی)
24 public class DeserializeExample {
25     public static void main(String[] args) {
26         try (ObjectInputStream ois = new ObjectInputStream(
27             new FileInputStream("person.ser"))) {
28             Person person = (Person) ois.readObject();
29             System.out.println("نام: " + person.getName()); // Ali
30             System.out.println("سن: " + person.getAge()); // 25
31             System.out.println("پسورد: " + person.getPassword()); // null
32         } catch (IOException | ClassNotFoundException e) {
33             e.printStackTrace();
34         }
35     }
36 }
37 }

```

سفرارشی سازی Serialization

```

1 class Person implements Serializable {
2     private static final long serialVersionUID = 1L;
3     private String name;
4     private int age;

```



```

5 private transient String password;
6
7 // نوشتن سفارشی
8 private void writeObject(ObjectOutputStream out) throws IOException {
9     out.defaultWriteObject();
10    out.writeUTF(password != null ? password : "default");
11 }
12
13 // خواندن سفارشی
14 private void readObject(ObjectInputStream in) throws IOException, ClassNotFoundException {
15     in.defaultReadObject();
16    password = in.readUTF();
17 }
18 }
19 }

```

۱.۵.۶ Console (ورودی از کاربر)

۱. Scanner (راحت‌ترین)

```

1 Scanner scanner = new Scanner(System.in);
2
3 System.out.print("نام: ");
4 String name = scanner.nextLine();
5
6 System.out.print("سن: ");
7 int age = scanner.nextInt();

```

```

8
9 System.out.print("حقوق: ");
10 double salary = scanner.nextDouble();
11
12 System.out.println("سال دارید " + age + " شما! " + name + " سلام");
13 scanner.close();

```

۲. BufferedReader (سریع تر)

```

1 try (BufferedReader reader = new BufferedReader(new InputStreamReader(System.in))) {
2     System.out.print("نام: ");
3     String name = reader.readLine();
4     System.out.print("سن: ");
5     int age = Integer.parseInt(reader.readLine());
6     System.out.println("سلام " + name);
7 } catch (IOException e) {
8     e.printStackTrace();
9 }

```

۳. Console (برای پسورد)

```

1 Console console = System.console();
2 if (console != null) {
3     String username = console.readLine("نام کاربری: ");
4     char[] password = console.readPassword("رمز عبور: ");
5     System.out.println("سلام " + username);
6 } else {

```

```
7 System.out.println("در دسترس نیست Console");
8 }
```

۱.۵.۷ ! سوالات مصاحبه‌ای پرتکرار

? سوال ۱: فرق Reader و InputStream چیه؟

جواب:

مقایسه Reader و InputStream در جاوا

ویژگی	InputStream	Reader
کاربرد	برای داده‌های باینری	برای داده‌های متنی
واحد خواندن	بایت (byte) می‌خواند	کاراکتر (char) می‌خواند
کدگذاری	ندارد	پشتیبانی از کدگذاری
مقدار بازگشتی read()	int read () برمی‌گردونه (0-255)	int read () برمی‌گردونه (0-65535)
مثال	تصویر، ویدیو	فایل TXT

? سوال ۲: فرق FileReader و FileInputStream چیه؟

جواب:

1 FileInputStream → برای فایل‌های **باینری** (تصویر، صدا، ویدیو)

2 FileReader → برای فایل‌های **متنی** (TXT, CSV)

? سوال ۳: BufferedReader چه مزیتی داره؟

جواب:

• بافرینگ: داده‌ها رو تکه‌تکه (بافر) می‌خونه، نه بایت‌به‌بایت

• سرعت بیشتر: به شدت سریع‌تر از FileReader معمولی

• متد readLine(): راحت‌ترین راه برای خواندن خط‌به‌خط

? سوال ۴: فرق static و transient در Serialization چیه؟

جواب:

مقایسه static و transient در جاوا

ویژگی	transient	static
محدوده	مخصوص هر شیء	متعلق به کلاس
ذخیره در Serialization	ذخیره نمیشه	ذخیره نمیشه
کاربرد	برای فیلدهای حساس (پسورد)	برای داده‌های اشتراکی
مقدار پس از Deserialize	null (یا مقدار پیش‌فرض)	مقدار کلاس (همان مقدار جاری)

? سوال ۵: Try-with-Resources چه مشکلی رو حل می‌کنه؟

جواب:

• قبلش برای بستن منابع کلی کد تکراری می‌نوشتیم

• مدیریت دستی استثناها دردسر داشت

• ممکن بود یادمون بره منابع رو ببندیم (Memory Leak)

• حالا خودکار و امن بسته میشن

؟ سوال ۶: IO و NIO چه فرقی دارن؟

جواب:

مقایسه IO و NIO در جاوا

ویژگی	IO	NIO
مبنای کار	Stream-based (یک طرفه)	Buffer-based (دوطرفه)
نوع عملکرد	Blocking (مسدود)	Non-blocking (غیرمسدود)
کاربرد مناسب	فایل‌های معمولی	سرورهای بزرگ
پیچیدگی API	ساده‌تر	قدرتمندتر
سرعت	کندتر	سریع‌تر

۱.۵.۸ خلاصه یک صفحه‌ای (دسته‌بندی کلاس‌های I/O در جاوا)

مفهوم	کلاس‌ها	کاربرد
Byte Input	FileInputStream, BufferedInputStream, DataInputStream	فایل‌های باینری
Byte Output	FileOutputStream, BufferedOutputStream, DataOutputStream	ذخیره باینری
Text Input	FileReader, BufferedReader, InputStreamReader	فایل‌های متنی
Text Output	FileWriter, BufferedWriter, PrintWriter	ذخیره متن
Serialization	ObjectInputStream, ObjectOutputStream	ذخیره اشیاء
NIO	Files, Paths, Path	روش مدرن I/O
User Input	Scanner, BufferedReader, Console	ورودی از کاربر